Devin Zhang

Location: United States

Email: dee.zhang.sde@gmail.com
Mobile: 458-272-8656

Summary

Senior Java Backend Engineer with 9+ years of experience designing and developing scalable, cloud-native microservices using **Java, Spring Boot, and RESTful APIs**. Strong expertise in building **event-driven systems with Kafka**, and integrating both **relational (PostgreSQL, MySQL)** and **NoSQL (MongoDB)** databases. Proficient in **JPA/Hibernate** for ORM, implementing robust **unit and integration tests** with **JUnit and Mockito**, and ensuring code quality through **CI/CD pipelines**. Hands-on experience with **Docker and Kubernetes** for container orchestration, and deploying production systems on **AWS (EC2, S3, RDS, Lambda)**. Familiar with backend **security best practices**, performance tuning, and delivering clean, maintainable, production-ready code.

Certificates

AWS Certified Solutions Architect - Associate

Programming Skills

- **Languages:** Java, Python, JavaScript, SQL, TypeScript
- **Frameworks & Technologies:** Spring Boot, Spring Cloud, Spring Security, Hibernate, JPA, JUnit, Mockito, Log4j, Maven, Feign, Flask, Django, Node.js, Express.js, React, Angular, Redux, Ant Design, Axios
- **Databases**: MySQL, PostgreSQL, Oracle, MongoDB, Redis, Cassandra, DynamoDB
- **Systems & APIs:** RESTful APIs, GraphQL, RabbitMQ, Kafka, Docker, Kubernetes, Swagger, SSO
- **Tools:** Git, GitHub Actions, Jenkins, Postman, Visual Studio, IntelliJ IDEA, Jupyter Notebooks, Bash Shell Scripting, Windows, Linux, Unix
- **Cloud & DevOps:** AWS (EC2, S3, RDS, Lambda, IAM, CloudWatch), Azure, Kubernetes, Ansible, Terraform
- **Testing & Monitoring:** TestNG, JUnit, Mockito, Cucumber, Selenium, Grafana, Splunk, CloudWatch

Experience

- **Databricks** — San Francisco, CA
  Software Engineer — *Oct. 2023 – Present*

  Worked on Databricks' Data Infrastructure team, responsible for building the ingestion and orchestration layer that powers real-time analytics for enterprise clients. The team handled multi-tenant, high-throughput data pipelines feeding into Spark-based computation and reporting systems. My role focused on building scalable ingestion services, optimizing Kafka-based streaming pipelines, and implementing robust coordination mechanisms using gRPC and Redis. I contributed to cross-AZ fault tolerance, observability, and production readiness of ingestion workflows at scale.

  - Developed ingestion and coordination microservices using Java 17, Spring Boot, and gRPC. Implemented multi-tenant request routing using interceptors and **ThreadLocal-based tenant context**. Defined .proto files, generated stubs with **protobuf-maven-plugin**, and handled **gRPC retries** via Resilience4j.
  - Optimized Kafka consumer and producer logic to handle 25M+ records/day, improving throughput by **42%**. Tuned **batch.size**, **linger.ms**, and enabled **snappy compression**; switched to async producer with callback. Improved consumer reliability with **manual offset commit** and fetch size tuning.
  - Designed and implemented Redis-based cache for schema and query plans, reducing Spark planning time by **35%**. Used **tenant-scoped cache keys**, TTLs, and schema versioning checks to avoid stale metadata. Profiling via Prometheus confirmed job planning improvements.
  - Implemented backpressure and batch buffering in Kafka ingestion to handle peak-load spikes. Added **lag-aware throttling** and in-memory **batch flushing** to stabilize ingestion during traffic surges. Reduced Kafka overloads and improved message throughput.
  - Containerized services with Docker and deployed to Kubernetes using Helm. Created Helm charts with **readiness/liveness probes**, resource limits, and environment-specific overrides. Supported versioned rollouts and rollback mechanisms.
  - Instrumented APIs with OpenTelemetry and Prometheus to monitor latency and error rates. Used **@Timed** and **custom @Gauge** metrics to track **tenant-level ingestion performance**. Built Grafana dashboards and **Slack alerting rules** for SLA violations.

- Introduced rate-limiting and token-bucket strategies for API burst control. Implemented **burst mitigation logic** to limit overload during load tests and high-concurrency workloads. Ensured ingestion service stayed responsive under pressure.
- Led migration from HTTP/1.1 REST to gRPC, improving internal service latency by **$\tilde{5}$0ms/request**. Rewrote internal calls using **gRPC clients**, generated from newly defined .proto contracts. Benchmarked improvement using **JMeter and Gatling**.
- Participated in on-call rotations and resolved ingestion queue deadlocks and schema race conditions. Helped write **partition reassignment tools** for stuck Kafka consumers. Fixed metadata cache bug by adding **Redis-based schema locks**.
- Wrote performance test suites using Gatling and JUnit to enable regression checks for each release. Simulated ingestion load patterns, validated **latency**, **throughput**, and **error rates** pre-deployment. Integrated into CI pipeline for weekly deployments.
- Tuned Spark executor configs and enabled checkpointing to reduce job failures. Adjusted **executor memory**, parallelism, and **shuffle partitions** to avoid OOM and retry loops. Enabled **state checkpointing** to recover from crash scenarios.
- Designed failover mechanism using Resilience4j circuit breaker and Redis fallback queue. Failed gRPC calls triggered **circuit breaking**, with jobs buffered in a **retry-safe Redis queue**. Ensured no data loss during coordination service outages.
- Migrated ingestion jobs from EC2-based ECS to **AWS Fargate**, reducing infra cost by **22%**. Created new **Fargate task definitions** and deployed across staging and production. Achieved faster startup and **per-task billing efficiency**.
- Benchmarked multi-region ingestion and proposed Kafka partition rebalancing to reduce **cross-AZ lag**. Identified **AZ-local Kafka routing** issues and used Kafka tools to reassign hot partitions. Improved region-specific performance metrics.
- Mentored two new hires and led weekly backend syncs focused on API contract stability and observability improvements. Documented ingestion workflows and query orchestration lifecycle, reducing onboarding time for new backend engineers by **40%**.

- **Stripe** San Francisco, CA
  Software Engineer *Sept. 2022 – Sept. 2023*

  Worked on Stripe's Payments Infrastructure team, responsible for internal ledger reconciliation, transaction validation, and fraud detection. The team processed 10M+ events daily and served as a critical backend layer ensuring financial correctness and compliance across global payments. My role focused on backend systems performance, fault-tolerant data pipelines, and integrating third-party fraud signals to improve validation coverage and resiliency.

  - Developed and optimized backend microservices using Java 17, Spring Boot, and Kafka, handling over 10M daily events. Built core transaction validation service to check balance, IP, and fraud risk; integrated with **Kafka** and downstream ledger system. Implemented **batch processing** and reduced DB call overhead using findAllById, cutting latency by **$\tilde{4}$0ms** during peak load.
  - Refactored ledger reconciliation logic to reduce processing latency by **40ms** under peak load. Cached historical entries in **Redis**, precomputed **hash digests**, and moved from single-record validation to **batch comparison**. These changes significantly reduced DB reads and improved reconciliation performance across manual and scheduled runs.
  - Implemented fraud detection signal ingestion pipelines from third-party APIs, improving validation coverage. Integrated with **Sift** and **MaxMind** using async **Spring WebClient**, handling timeouts with **Resilience4j circuit breakers**. Persisted raw fraud scores for auditability and blocked high-risk transactions or routed them to **RabbitMQ** for manual review.
  - Built async retry mechanisms for transient transaction failures using **RabbitMQ** and **Backoff strategies**. On failure, published metadata-rich messages to **retry queues**, using **exponential backoff with jitter** to avoid overload. Routed exhausted retries to DLQ, improving recovery and decoupling critical services during traffic spikes or outages.
  - Added Redis caching for frequently accessed account balance snapshots with TTL and stateness checks, reducing DB read pressure by **60%**. Cached account balance under keys like balance:{accountId} with TTL and staleness checks. Reduced pressure on **PostgreSQL ledger DB** during high-read operations like pre-transaction balance checks.
  - Developed internal APIs for ledger lookups and audit trails with **role-based access control**. Built secure internal endpoints using **Spring Security** and **JWT-based RBAC** for finance/compliance teams. Added @PreAuthorize role restrictions; enabled **self-serve audit trail access**, reducing manual requests by **60%+**.
  - Used **PostgreSQL** and **Flyway** for schema versioning and optimized large table reads using partitioning. Implemented **monthly range partitioning** on the ledger table and indexed each partition. Improved performance for long-range queries and ensured **safe schema migrations** across environments with Flyway.
  - Contributed to containerization and deployment workflows using **Docker** and **Kubernetes**. Maintained Dockerfiles with **base image tuning**, JVM config, and proper entry points for each ledger service. Adjusted **Kubernetes resource limits** and probes for container stability; collaborated with platform/SRE team.

- Added custom Micrometer metrics like transactions.dropped and exposed to Datadog; built dashboards and alerts. Defined service-level metrics using **Micrometer**, exported them with **Datadog registry**, and visualized in dashboards. Created alert rules for transaction drop spikes, latency percentiles, and release annotations for easier root-cause tracing.
- Collaborated with QA team to write regression tests simulating race conditions, duplicate transactions, and bad metadata. Developed **JUnit** and **Mockito** tests covering edge cases like replayed IDs and malformed payloads. Helped prevent ledger inconsistencies and reduced post-release incidents with pre-deploy test coverage.
- Participated in incident review processes, analyzed retry spikes with Datadog logs and proposed retry logic improvements that reduced transaction retries by **22%**. Identified excessive retries due to transient DB issues; added **retry caps**, backoff with jitter, and DLQ fallback. Reduced retry volume and improved system responsiveness under transient failure scenarios.
- Documented ledger processing flow and designed onboarding guide for future backend interns. Wrote system architecture docs, request lifecycle walkthroughs, and common error patterns. Reduced onboarding time by **40%** and standardized new-hire ramp-up with a self-guided guide adopted by the team.

- **NextRoll** San Francisco, CA
  Software Engineer *Jul. 2018 – Aug. 2022*

  Worked on NextRoll's Ads Infrastructure team, responsible for real-time ad targeting, campaign ranking, and delivery optimization across multiple platforms. Our backend system processed over 5 billion events daily and powered low-latency, high-reliability ad delivery. As a backend engineer, I focused on building scalable Java microservices, optimizing Kafka-Flink pipelines, integrating Redis and MySQL for low-latency access, and improving observability for ML-driven ad delivery systems.

  - Developed and maintained core backend services using **Java 11**, **Spring Boot**, and **Kafka**, processing over **5 million events per day**. Built userInteractionIngestionService to enrich Kafka events with **session metadata from Redis**, reducing lookup latency by **40%**. Published enriched events to downstream systems for **ranking and campaign selection** in real time.
  - Designed RESTful APIs for ad targeting microservices, improving request/response throughput by **35%**. Migrated legacy gRPC APIs to **Spring Boot REST endpoints** for better compatibility across teams and tooling. Applied **stream-based filtering** on user profiles and geo-data to return matched ad campaigns.
  - Optimized asynchronous data pipelines with **Kafka** and **Flink**, reducing ad targeting latency from **180ms to 90ms**. Maintained Flink jobs with **tumbling windows** and stream transformations for campaign metrics. Tuned Kafka buffer sizes and Flink checkpoint settings to achieve near real-time responsiveness.
  - Integrated **Redis** as a low-latency cache layer to store recent user-ad interaction history, Stored last 10 ad impressions/clicks per user with **10-minute TTL** to support frequency capping. Replaced slow DB queries with **Redis list lookups**, reducing lookup latency by **$\tilde{9}$0ms**.
  - Refactored monolithic components into containerized microservices with **Docker** and deployed using **Kubernetes**, decoupled impression tracking logic into a standalone service, deployed with **Helm** and **ConfigMaps**. Reduced deployment time from **20 minutes to 5 minutes**, with improved testability and scaling.
  - Led the migration of several backend services to **gRPC**, enhancing observability and request tracing. Rewrote service contracts using **Protocol Buffers** and implemented **OpenTelemetry-based tracing**. Integrated with **Datadog APM**, enabling full request-span visibility and latency debugging.
  - Tuned **MySQL** queries and indexing strategies to reduce read contention on impression-tracking tables. Analyzed slow queries using EXPLAIN and SHOW PROFILE; added **composite indexes** like (user_id, ad_id, timestamp). Reduced **P95 read latency by $\tilde{4}$5%**, especially during peak query hours.
  - Built real-time metrics dashboards with **Prometheus** and **Grafana**, enabling early detection of delivery bottlenecks. Monitored Kafka lag, Redis hit rate, gRPC latency, and Flink processing time via **Grafana panels**. Enabled faster diagnosis of job lag and API timeouts using visual alerts.
  - Collaborated with ML engineers to expose real-time backend signals via Kafka and APIs, delivered features like user-ad interaction summaries, device info, and geo-data to ML scoring systems. Maintained **shared schemas and contracts**, and joined weekly model integration reviews.
  - Implemented rate limiting and circuit breaker logic using **Resilience4j**, improving service stability during traffic spikes. Used **token bucket strategy** for per-client rate limits, configured via Spring Cloud Config. Applied **circuit breaking** for external user profile and geo-IP services to prevent cascading failures.
  - Conducted load testing with **JMeter**, verifying system performance under simulated traffic of 1M RPS. Ran **distributed JMeter tests**, monitored latency, error rates, and system throughput under pressure. Identified Redis hotkeys and optimized thread pools to maintain **low error rate and stable latency**.
  - Participated in weekly SRE on-call rotation to ensure 99.99% availability of the ad delivery subsystem. Triaged alerts via **Prometheus + PagerDuty**; mitigated Redis hotkey and geo-lookup cache penetration issues. Used **fallbacks** and null caching to maintain availability when upstream data was delayed or missing.

- Conducted code reviews for junior engineers, focusing on readability, testability, and performance. Reviewed critical paths, suggested **unit test coverage**, and promoted reusable interface design. Helped reduce post-merge issues and improved PR turnaround for new contributors.

- **Bloomberg China** Beijing
  Software Engineer *Jun. 2016 – Jun. 2018*

  Worked on Bloomberg's Beijing Engineering team, contributing to backend systems for Asia-Pacific (APAC) market data ingestion and internal talent-matching tools. The market data pipeline consumed proprietary equity feeds, parsed, enriched, and published them to internal services for downstream analytics and real-time dashboards. My role focused on implementing multi-threaded ingestion services, improving data validation reliability, and building APIs to support internal HR applications. I also contributed to production monitoring, feed quality alerting, and developer documentation.

  - Developed ingestion services using Java and Spring Boot to subscribe to Bloomberg's SDK, decode binary APAC equity feeds, validate metadata, and publish structured output to **Kafka** and internal APIs. Validated ticker/exchange codes against internal databases and filtered out invalid records to ensure data quality and avoid polluting downstream analytics.
  - Built candidate-matching services and RESTful APIs to support LinkedIn-style internal HR tools, improving query latency by **25%** via **SQL joins**, subqueries, and composite indexing. Enabled integration with internal dashboards and HR workflows through well-documented endpoints using **Spring MVC**.
  - Used Java's **ExecutorService** and concurrent queues to implement multi-threaded processing of equity feed bursts, improving ingestion throughput by **40%** during market open. Designed fixed-thread worker pools to ensure stable processing under high-volume conditions.
  - Implemented **Kafka consumers** with Spring Kafka for downstream data flow, supporting **manual ack**, JSON deserialization with Jackson, and **DLQ routing** for failed messages. Used custom error handlers and resilience logic to handle malformed or incomplete feed data.
  - Introduced Redis caching for high-frequency metadata lookups and batched DB access, reducing **redundant queries** by 30% and improving average response time for lookup-heavy services. Applied TTL to control staleness and cache invalidation behavior.
  - Automated data sync jobs using **Quartz Scheduler** to fetch and upsert candidate profiles nightly from internal HR systems into local databases. Improved data freshness for internal matching tools without manual triggers.
  - Designed alerting rules in Bloomberg Terminal's internal engine to catch ticker validation or price field errors, reducing unnoticed feed issues by **50%**. Triggered team alerts when error counts crossed defined thresholds within time windows
  - Wrote unit and integration tests with **JUnit 5** and **Mockito**, mocking SDK feeds, Kafka producers, and DAO layers; ensured **test coverage over 85%**, verified with **JaCoCo**. Focused on validating parsing logic, enrichment correctness, and API response behavior.
  - Monitored ingestion health via **Prometheus and Grafana**, visualizing feed lag, success/error rates, and system throughput. Debugged failed data issues using **Splunk** and Bloomberg's internal message-trace logs.
  - Authored internal documentation explaining **ticker normalization**, ingestion workflow, and common error patterns to assist future interns and engineers. Also added **Swagger-based API documentation**, reducing onboarding and integration time for new developers.
  - Learned basic **FIX protocol** structure used in financial data transfer and contributed to debugging malformed messages

Education

Oregon State University, Corvallis, OR
Master of Engineering in Computer Science

Henan University of Engineering, Henan, China
Bachelor of Science in Computer and Information Science